

Multi-Agent Deep Reinforcement Learning for Autonomous Navigation and Collision Avoidance of Unmanned Ground Vehicles

Sajan Bhujel

M.S. Student

University of North Dakota
Grand Forks, ND 58202, USA
sajan.bhujel@und.edu

Abstract—Autonomy in multi-robot systems requires reliable decision-making under shared, dynamic, and uncertain operating conditions, where each robot must navigate toward its mission objective while maintaining safety with respect to obstacles, boundaries, and neighboring agents. This problem becomes more difficult when the robots operate with local observations, since the motion of one robot changes the state experienced by the others. To address this challenge, this paper presents a multi-agent deep reinforcement learning framework for autonomous navigation and collision avoidance of unmanned ground vehicles. The navigation task is formulated as a multi-agent continuous-control problem for differential-drive robots using LiDAR-based perception, goal-relative information, neighboring-robot information, and velocity states. A Multi-Agent Deep Deterministic Policy Gradient (MADDPG) controller is developed under a Centralized Training with Decentralized Execution (CTDE) framework. During training, centralized critics use the global multi-agent state and joint action, while each actor executes using only local observations. The framework is evaluated in a four-robot ROS 2/Gazebo simulation environment with continuous linear and angular velocity commands. Training behavior, policy execution, and saliency-based feature analysis are used to assess the learned navigation policy. The results show that the proposed framework can generate goal-directed and collision-aware navigation behavior for multiple unmanned ground vehicles in simulation.

Keywords: Multi-agent reinforcement learning, differential-drive robot, Gazebo, autonomous navigation, centralized training decentralized execution.

I. INTRODUCTION

Autonomous robotic systems are increasingly important for aerospace, defense, emergency response, logistics, and planetary exploration applications where vehicles must operate with limited human intervention [1], [2]. A central challenge in these systems is autonomous navigation, where a vehicle must make real-time decisions to reach mission objectives while maintaining safety in constrained and uncertain environments [3]. For multi-robot systems, this challenge becomes more complex because each vehicle must account not only for obstacles and environmental boundaries, but also for the motion and behavior of other agents operating in the same shared space.

Traditional navigation methods commonly rely on pre-defined planners, local feedback controllers, and rule-based

collision avoidance strategies [4], [5]. Although these approaches can perform well in structured environments, they often require extensive hand tuning and may not adapt effectively to complex multi-agent interactions. Learning-based control provides an alternative framework in which robots learn navigation policies directly from interaction with the environment. Reinforcement learning is particularly attractive for this problem because it allows agents to optimize long-term task performance through trial-and-error experience rather than relying entirely on manually designed control rules [6], [7].

This work investigates a multi-agent deep reinforcement learning approach for autonomous navigation and collision avoidance of unmanned ground vehicles. The vehicles are modeled as differential-drive mobile robots equipped with LiDAR-based perception. Each robot receives local observations that include range information, pose information, goal-relative features, neighboring-agent information, and velocity states. The control policy outputs continuous linear and angular velocity commands, making the problem a continuous-control multi-agent decision-making task.

To address the non-stationarity introduced by multiple learning agents, this work adopts Multi-Agent Deep Deterministic Policy Gradient (MADDPG) with Centralized Training and Decentralized Execution (CTDE) [8]. During training, centralized critics use the global multi-agent state and joint action to evaluate policy behavior, while during execution each robot selects actions using only its local observation. This structure is suitable for autonomous robotic systems because it improves learning stability during training while preserving decentralized onboard decision-making during deployment. The main contributions of this work are summarized as follows:

- 1) A multi-agent autonomous navigation framework is developed for differential-drive unmanned ground vehicles in a physics-based simulation environment, where each robot uses local observations and continuous velocity commands.
- 2) The navigation and collision-avoidance task is formulated as a multi-agent continuous-control reinforcement learning problem, and a MADDPG-based CTDE control architecture is implemented for centralized critic training

and decentralized actor execution.

- 3) A four-robot case study is conducted to evaluate goal-directed navigation, obstacle avoidance, and inter-agent collision avoidance, with training performance and saliency-based policy interpretation used to analyze the learned behavior.

II. PROBLEM FORMULATION

A. Multi-Robot System Description

Consider a system of N autonomous differential-drive mobile robots operating in a bounded planar environment. The set of robots is denoted as

$$\mathcal{N} = \{1, 2, \dots, N\}. \quad (1)$$

Each robot is treated as an individual learning agent. The pose of the n -th robot at time t is defined as

$$\mathbf{x}_n(t) = \begin{bmatrix} x_n(t) \\ y_n(t) \\ \psi_n(t) \end{bmatrix}, \quad n \in \mathcal{N}, \quad (2)$$

where $x_n(t)$ and $y_n(t)$ are the global planar coordinates of robot n , and $\psi_n(t)$ is its yaw angle. The planar position of robot n is

$$\mathbf{p}_n(t) = \begin{bmatrix} x_n(t) \\ y_n(t) \end{bmatrix}. \quad (3)$$

Each robot has an assigned goal position

$$\mathbf{g}_n = \begin{bmatrix} x_{g,n} \\ y_{g,n} \end{bmatrix}. \quad (4)$$

The distance between robot n and its assigned goal is

$$d_n(t) = \|\mathbf{p}_n(t) - \mathbf{g}_n\|_2. \quad (5)$$

The objective of each robot is to reach its assigned goal while avoiding static obstacles, arena boundaries, and other robots. The multi-robot navigation problem is challenging because the motion of each robot changes the environment experienced by the remaining robots. Therefore, the problem is formulated as a multi-agent reinforcement learning problem rather than a collection of independent single-agent control problems.

B. Differential-Drive Robot Kinematics

Each robot follows differential-drive kinematics. For the n -th robot, the continuous-time motion model is given by

$$\dot{x}_n(t) = v_n(t) \cos \psi_n(t), \quad (6)$$

$$\dot{y}_n(t) = v_n(t) \sin \psi_n(t), \quad (7)$$

$$\dot{\psi}_n(t) = \omega_n(t), \quad (8)$$

where $v_n(t)$ is the linear velocity command and $\omega_n(t)$ is the angular velocity command of robot n . The control input for robot n is defined as

$$\mathbf{u}_n(t) = \begin{bmatrix} v_n(t) \\ \omega_n(t) \end{bmatrix}. \quad (9)$$

The velocity commands are bounded by the physical limits of the robot:

$$v_n(t) \in [v_{\min}, v_{\max}], \quad (10)$$

$$\omega_n(t) \in [-\omega_{\max}, \omega_{\max}]. \quad (11)$$

Using a discrete-time approximation with sampling interval Δt , the kinematic update for robot n can be written as

$$x_n(t + \Delta t) = x_n(t) + \Delta t v_n(t) \cos \psi_n(t), \quad (12)$$

$$y_n(t + \Delta t) = y_n(t) + \Delta t v_n(t) \sin \psi_n(t), \quad (13)$$

$$\psi_n(t + \Delta t) = \psi_n(t) + \Delta t \omega_n(t). \quad (14)$$

For a differential-drive robot with wheel radius r_w and wheel separation L , the relationship between the wheel angular velocities and body-frame velocity commands is

$$v_n(t) = \frac{r_w}{2} (\omega_{R,n}(t) + \omega_{L,n}(t)), \quad (15)$$

$$\omega_n(t) = \frac{r_w}{L} (\omega_{R,n}(t) - \omega_{L,n}(t)), \quad (16)$$

where $\omega_{R,n}(t)$ and $\omega_{L,n}(t)$ are the right and left wheel angular velocities of robot n , respectively. In this work, the reinforcement learning policy directly outputs the body-level velocity command $\mathbf{u}_n(t) = [v_n(t), \omega_n(t)]^T$, which is then applied to the differential-drive robot in simulation.

III. METHODOLOGY

A. MDP Formulation

The multi-robot navigation task is formulated as a multi-agent Markov decision process [9]. The complete multi-agent system is represented as

$$\mathcal{M} = (\mathcal{S}, \{\mathcal{A}_n\}_{n=1}^N, P, \{R_n\}_{n=1}^N, \gamma), \quad (17)$$

where $\mathcal{S}$ is the global state space, $\mathcal{A}_n$ is the action space of robot n , P is the transition model, R_n is the reward function for robot n , and γ is the discount factor.

1) *State*: Each robot receives a local observation vector constructed from its sensor readings, pose information, goal-relative information, relative positions of other robots, and current velocity command. The local observation of robot n is denoted as

$$\mathbf{o}_n(t) \in \mathcal{O}_n. \quad (18)$$

Let $\mathbf{l}_n(t)$ denote the processed LiDAR observation of robot n . The goal-relative angle is defined as

$$\alpha_n(t) = \text{wrap} \left[\tan^{-1} \left(\frac{y_{g,n} - y_n(t)}{x_{g,n} - x_n(t)} \right) - \psi_n(t) \right], \quad (19)$$

where $\text{wrap}(\cdot)$ maps the angle to the principal interval. The relative position of robot m with respect to robot n is

$$\mathbf{r}_{nm}(t) = \begin{bmatrix} x_m(t) - x_n(t) \\ y_m(t) - y_n(t) \end{bmatrix}, \quad m \neq n. \quad (20)$$

The local observation of robot n is written as

$$\mathbf{o}_n(t) = [\mathbf{l}_n(t), x_n(t), y_n(t), \psi_n(t), d_n(t), \alpha_n(t), \{\mathbf{r}_{nm}(t)\}_{m \neq n}, v_n(t), \omega_n(t)]. \quad (21)$$

The centralized state used during critic training is formed by concatenating the local observations of all robots:

$$\mathbf{s}(t) = [\mathbf{o}_1(t), \mathbf{o}_2(t), \dots, \mathbf{o}_N(t)]. \quad (22)$$

2) *Action*: The action of robot n is the continuous velocity command

$$\mathbf{a}_n(t) = \begin{bmatrix} v_n(t) \\ \omega_n(t) \end{bmatrix} \in \mathcal{A}_n. \quad (23)$$

The joint action of all robots is

$$\mathbf{a}(t) = [\mathbf{a}_1(t), \mathbf{a}_2(t), \dots, \mathbf{a}_N(t)]. \quad (24)$$

The learned actor policy for robot n maps its local observation to its continuous action:

$$\mathbf{a}_n(t) = \mu_n(\mathbf{o}_n(t); \theta_n^\mu), \quad (25)$$

where θ_n^μ denotes the parameters of the actor network for robot n .

3) *Transition Model*: The transition model describes how the multi-robot system evolves after applying the joint action. It is represented as

$$\mathbf{s}(t + \Delta t) \sim P(\mathbf{s}(t + \Delta t) | \mathbf{s}(t), \mathbf{a}_1(t), \dots, \mathbf{a}_N(t)). \quad (26)$$

In this project, the transition model is not known analytically. Instead, transitions are generated by the robot simulation environment. At each time step, the joint action is applied to the robots, the simulator updates the robot poses and sensor readings, and the next state is observed. A sampled transition stored during training is

$$(\mathbf{s}(t), \mathbf{O}(t), \mathbf{a}(t), \mathbf{r}(t), \mathbf{s}(t + \Delta t), \mathbf{O}(t + \Delta t), \mathbf{d}(t)), \quad (27)$$

where

$$\mathbf{O}(t) = [\mathbf{o}_1(t), \dots, \mathbf{o}_N(t)], \quad (28)$$

$$\mathbf{r}(t) = [r_1(t), \dots, r_N(t)], \quad (29)$$

and $\mathbf{d}(t)$ is the vector of terminal flags.

4) *Optimization Formulation*: The objective of each robot is to learn a policy that maximizes its expected discounted return:

$$J_n(\theta_n^\mu) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t r_n(t) \right]. \quad (30)$$

For the complete multi-agent system, the cooperative objective can be written as

$$J(\theta^\mu) = \frac{1}{N} \sum_{n=1}^N J_n(\theta_n^\mu), \quad (31)$$

where

$$\theta^\mu = \{\theta_1^\mu, \theta_2^\mu, \dots, \theta_N^\mu\}. \quad (32)$$

The optimization problem is therefore

$$\theta^{\mu*} = \arg \max_{\theta^\mu} J(\theta^\mu). \quad (33)$$

This objective encourages all robots to learn policies that produce goal-directed and collision-aware navigation behavior.

5) *Reward*: The reward function for robot n is designed to encourage progress toward the goal while penalizing unsafe or inefficient behavior. The progress term is defined as

$$\Delta d_n(t) = d_n(t - \Delta t) - d_n(t). \quad (34)$$

The general reward function is

$$\begin{aligned} r_n(t) = & w_p \Delta d_n(t) - w_d d_n(t) - w_s \\ & - w_{\text{near}} I_{\text{near},n}(t) - w_{\text{idle}} I_{\text{idle},n}(t) \\ & + w_{\text{goal}} I_{\text{goal},n}(t) - w_{\text{col}} I_{\text{col},n}(t) \\ & - w_{\text{max}} I_{\text{max},n}(t). \end{aligned} \quad (35)$$

where w_p , w_d , w_s , w_{near} , w_{idle} , w_{goal} , w_{col} , and w_{max} are reward weighting parameters. The indicator variables are defined as follows:

$$I_{\text{goal},n}(t) = \begin{cases} 1, & d_n(t) < r_g, \\ 0, & \text{otherwise}, \end{cases} \quad (36)$$

The indicator variables are defined as

$$\begin{aligned} I_n^{\text{goal}}(t) &= \begin{cases} 1, & d_n(t) < r_g, \\ 0, & \text{otherwise}, \end{cases} \\ I_n^{\text{col}}(t) &= \begin{cases} 1, & \text{collision}, \\ 0, & \text{otherwise}, \end{cases} \\ I_n^{\text{near}}(t) &= \begin{cases} 1, & \text{near obstacle}, \\ 0, & \text{otherwise}, \end{cases} \\ I_n^{\text{idle}}(t) &= \begin{cases} 1, & \text{nearly stationary}, \\ 0, & \text{otherwise}, \end{cases} \\ I_n^{\text{max}}(t) &= \begin{cases} 1, & t \geq T, \\ 0, & \text{otherwise}. \end{cases} \end{aligned} \quad (37)$$

This reward structure provides dense feedback through the progress and distance terms, while goal completion and collision events provide terminal feedback.

B. MADDPG with CTDE Control Approach

The control method used in this work is Multi-Agent Deep Deterministic Policy Gradient (MADDPG) under the Centralized Training with Decentralized Execution framework [10]. MADDPG extends deterministic actor-critic learning to multi-agent systems by assigning each robot its own actor and critic networks. Each robot n has an actor network

$$\mu_n(\mathbf{o}_n; \theta_n^\mu), \quad (38)$$

which maps local observations to continuous control actions. Each robot also has a centralized critic network

$$Q_n(\mathbf{s}, \mathbf{a}_1, \dots, \mathbf{a}_N; \theta_n^Q), \quad (39)$$

which estimates the action-value function for robot n using the global state and joint action. During training, the critic uses centralized information:

$$Q_n = Q_n(\mathbf{s}, \mathbf{a}_1, \dots, \mathbf{a}_N). \quad (40)$$

During execution, the actor uses only local information:

$$\mathbf{a}_n = \mu_n(\mathbf{o}_n). \quad (41)$$

This structure reduces the non-stationarity of multi-agent learning during training while preserving decentralized execution during testing.

1) *Actor Network*: The actor network for robot n is represented as

$$\mathbf{h}_n^\mu = f_\mu(\mathbf{o}_n; \theta_n^\mu), \quad (42)$$

where $f_\mu(\cdot)$ denotes the hidden layers of the actor network. The final output is passed through a bounded activation function:

$$\tilde{\mathbf{a}}_n = \tanh(W_n^\mu \mathbf{h}_n^\mu + \mathbf{b}_n^\mu). \quad (43)$$

The bounded output is scaled to the physical action limits:

$$v_n = \mathcal{S}_v(\tilde{a}_{v,n}), \quad (44)$$

$$\omega_n = \mathcal{S}_\omega(\tilde{a}_{\omega,n}), \quad (45)$$

where $\mathcal{S}_v(\cdot)$ and $\mathcal{S}_\omega(\cdot)$ are scaling functions that map neural network outputs to admissible linear and angular velocity ranges.

2) *Critic Network*: The critic network for robot n receives the centralized state and joint action:

$$\mathbf{z} = [\mathbf{s}, \mathbf{a}_1, \dots, \mathbf{a}_N]. \quad (46)$$

The critic output is

$$Q_n = f_Q(\mathbf{z}; \theta_n^Q), \quad (47)$$

where $f_Q(\cdot)$ denotes the critic network. For each robot n , target actions are generated using the target actor networks:

$$\mathbf{a}'_m = \mu'_m(\mathbf{o}'_m; \theta'_m), \quad m \in \mathcal{N}. \quad (48)$$

The target value for robot n is

$$y_n = r_n + \gamma(1 - d_n)Q'_n(\mathbf{s}', \mathbf{a}'_1, \dots, \mathbf{a}'_N; \theta_n^{Q'}), \quad (49)$$

where d_n is the terminal flag, Q'_n is the target critic, and μ'_m is the target actor for robot m . The critic loss is

$$L_n^Q = \mathbb{E}_{\mathcal{D}} \left[(Q_n(\mathbf{s}, \mathbf{a}_1, \dots, \mathbf{a}_N) - y_n)^2 \right], \quad (50)$$

where $\mathcal{D}$ is the replay buffer.

3) *Actor Update*: The actor for robot n is updated by maximizing the centralized critic value. Equivalently, the loss minimized during implementation is

$$L_n^\mu = -\mathbb{E}_{\mathcal{D}} [Q_n(\mathbf{s}, \mathbf{a}_1, \dots, \mu_n(\mathbf{o}_n), \dots, \mathbf{a}_N)]. \quad (51)$$

The deterministic policy gradient for robot n can be written as

$$\nabla_{\theta_n^\mu} J_n = \mathbb{E}_{\mathcal{D}} \left[\nabla_{\mathbf{a}_n} Q_n(\mathbf{s}, \mathbf{a}_1, \dots, \mathbf{a}_N) \Big|_{\mathbf{a}_n = \mu_n(\mathbf{o}_n)} \nabla_{\theta_n^\mu} \mu_n(\mathbf{o}_n; \theta_n^\mu) \right]. \quad (52)$$

Algorithm 1 MADDPG Training for Multi-Agent Robot Navigation

```

1: Input: number of robots  $N$ , episodes  $M$ , episode length  $T$ ,
   replay buffer  $\mathcal{D}$ , batch size  $B$ , discount factor  $\gamma$ , soft update
   factor  $\tau$ 
2: Initialize actor networks  $\mu_n(\mathbf{o}_n; \theta_n^\mu)$  and critic networks
    $Q_n(\mathbf{s}, \mathbf{a}_1, \dots, \mathbf{a}_N; \theta_n^Q)$  for all  $n \in \{1, \dots, N\}$ 
3: Initialize target networks  $\mu'_n$  and  $Q'_n$  with  $\theta_n^{\mu'} \leftarrow \theta_n^\mu$  and  $\theta_n^{Q'} \leftarrow \theta_n^Q$ 
4: for episode = 1 to  $M$  do
5:   Reset the simulation environment
6:   Obtain local observations  $\mathbf{o}_1, \dots, \mathbf{o}_N$ 
7:   Construct global state  $\mathbf{s} = [\mathbf{o}_1, \dots, \mathbf{o}_N]$ 
8:   for  $t = 1$  to  $T$  do
9:     for robot  $n = 1$  to  $N$  do
10:      Select action with exploration:
11:       $\mathbf{a}_n = \mu_n(\mathbf{o}_n; \theta_n^\mu) + \epsilon_n$ 
12:      Clip  $\mathbf{a}_n$  to satisfy velocity limits
13:    end for
14:    Execute joint action  $\mathbf{a} = [\mathbf{a}_1, \dots, \mathbf{a}_N]$ 
15:    Observe rewards  $\mathbf{r}$ , next observations  $\mathbf{o}'_1, \dots, \mathbf{o}'_N$ , and done
    flags  $\mathbf{d}$ 
16:    Construct next global state  $\mathbf{s}' = [\mathbf{o}'_1, \dots, \mathbf{o}'_N]$ 
17:    Store  $(\mathbf{s}, \mathbf{o}, \mathbf{a}, \mathbf{r}, \mathbf{s}', \mathbf{o}', \mathbf{d})$  in  $\mathcal{D}$ 
18:    if  $|\mathcal{D}| \geq B$  then
19:      Sample a minibatch from  $\mathcal{D}$ 
20:      for robot  $n = 1$  to  $N$  do
21:        Compute target actions  $\mathbf{a}'_m = \mu'_m(\mathbf{o}'_m)$  for all  $m$ 
22:        Compute target:
23:         $y_n = r_n + \gamma(1 - d_n)Q'_n(\mathbf{s}', \mathbf{a}'_1, \dots, \mathbf{a}'_N)$ 
24:        Update critic by minimizing:
25:         $L_n^Q = \frac{1}{B} \sum (Q_n(\mathbf{s}, \mathbf{a}_1, \dots, \mathbf{a}_N) - y_n)^2$ 
26:        Update actor by minimizing:
27:         $L_n^\mu = -\frac{1}{B} \sum Q_n(\mathbf{s}, \mathbf{a}_1, \dots, \mu_n(\mathbf{o}_n), \dots, \mathbf{a}_N)$ 
28:        Soft update target networks:
29:         $\theta_n^{\mu'} \leftarrow \tau \theta_n^\mu + (1 - \tau) \theta_n^{\mu'}$ 
30:         $\theta_n^{Q'} \leftarrow \tau \theta_n^Q + (1 - \tau) \theta_n^{Q'}$ 
31:      end for
32:    end if
33:    Set  $\mathbf{s} \leftarrow \mathbf{s}'$  and  $\mathbf{o}_n \leftarrow \mathbf{o}'_n$  for all robots
34:  end for
35:  Save rewards, done counts, and model checkpoints
36: end for

```

4) *Target Network Update*: Each actor and critic has a corresponding target network. The target actor and critic parameters are updated using soft updates:

$$\theta_n^{\mu'} \leftarrow \tau \theta_n^\mu + (1 - \tau) \theta_n^{\mu'}, \quad (53)$$

$$\theta_n^{Q'} \leftarrow \tau \theta_n^Q + (1 - \tau) \theta_n^{Q'}. \quad (54)$$

The soft update improves training stability by preventing rapid changes in the target values.

IV. CASE STUDY

A. Simulation Environment

The case study is performed in a Gazebo simulation environment. Four EduBot differential-drive robots are placed in a bounded arena with static obstacles. Each robot has a unique start pose and goal position. The robots receive 2D LiDAR observations and send velocity commands through

TABLE I: Start and goal positions for the four-robot case study.

Robot	x_0 (m)	y_0 (m)	ψ_0 (rad)	(x_g, y_g) (m)
1	-2.0	2.0	-0.661	(2.5, -1.5)
2	2.5	2.5	-2.356	(-2.0, -2.0)
3	-1.0	-3.0	1.571	(-1.0, 3.0)
4	3.0	0.0	2.896	(-1.0, 1.0)

ROS 2 topics. The Gazebo robot model uses a differential-drive system with left and right wheel joints. A LiDAR sensor provides 360 range samples over a horizontal field of view from $-\pi$ to π . The LiDAR range is limited between 0.05 m and 8.0 m. The LiDAR data are downsampled into 100 bins before being used by the learning policy. The four-robot navigation task uses the start and goal configurations shown in Table I. Each robot starts from a different region of the arena and is assigned a different goal.

The main simulation and training parameters are summarized in Table II.

TABLE II: Main simulation and learning parameters.

Parameter	Value
Number of robots	2 and 4
Robot type	EduBot differential-drive robot
Simulation platform	ROS 2 and Gazebo
Action dimension per robot	2
Action components	Linear velocity and angular velocity
Maximum linear velocity	0.35 m/s
Maximum angular velocity	0.5 rad/s
LiDAR samples	360
Downsampled LiDAR bins	100
Maximum LiDAR range	8.0 m
Goal radius	0.4 m
Obstacle collision distance	0.18 m
Near-obstacle distance	0.3 m
Robot-robot collision distance	0.35 m
Maximum episode length	500 steps
Training episodes	1000
Replay buffer size	200000
Batch size	256
Discount factor γ	0.99
Soft update factor τ	0.01
Actor learning rate	10^{-4}
Critic learning rate	10^{-3}

B. Training Reward

The training reward curve is used to analyze whether the agents improve over training episodes. The average reward is computed as

$$\bar{R}_{ep} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T r_i(t). \quad (55)$$

The expected learning behavior is that early episodes produce lower reward because the robots explore inefficiently and may collide or fail to reach the goal. As training progresses, the reward should improve if the actors learn goal-directed motion and the critics provide useful gradients for policy improvement.

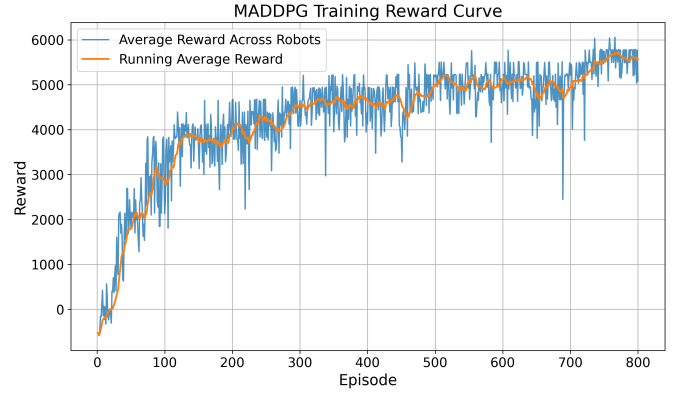


Fig. 1: MADDPG training reward curve for the multi-agent Gazebo navigation task. The curve shows the average reward trend during training.

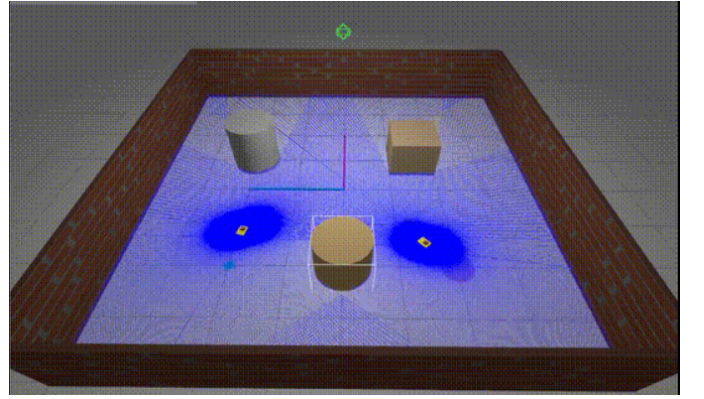


Fig. 2: Two-robot Gazebo simulation case study for multi-agent differential-drive robot navigation.

After training, the actor networks are loaded and tested without exploration noise. During testing, each robot selects actions using only its local observation:

$$\mathbf{a}_i(t) = \mu_i(\mathbf{o}_i(t)). \quad (56)$$

This testing setup evaluates decentralized execution. The critic networks are not required during execution.

Two simulation case studies were conducted to evaluate the proposed multi-agent navigation framework. The first case study considers a two-robot navigation scenario, as shown in Fig. 2. In this case, two differential-drive robots are placed in the bounded Gazebo arena with static cylindrical and box-shaped obstacles. The blue LiDAR rays show the local sensing field used by each robot to detect nearby obstacles and arena boundaries. This case study is used to verify the basic multi-agent navigation behavior, where each robot must move toward its assigned goal while avoiding obstacle collision and inter-robot conflict.

The second case study extends the environment to a four-robot navigation scenario, as shown in Fig. 3. This scenario is more challenging because the number of moving agents increases, which also increases the possibility of inter-robot interaction and collision. Each robot uses its local observation,

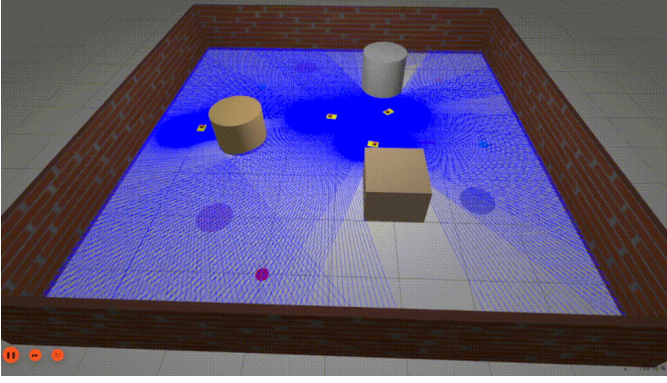


Fig. 3: Four-robot Gazebo simulation case study with static obstacles, boundary walls, and LiDAR visualization for collision-aware navigation.

including LiDAR-based range information and goal-relative features, to generate continuous linear and angular velocity commands. This four-robot case study is used to evaluate whether the MADDPG-based CTDE framework can scale from a simpler two-agent setup to a more complex multi-agent navigation task while maintaining collision-aware behavior.

V. DISCUSSION

A. Saliency Analysis

To interpret the learned policy, a saliency analysis is used to estimate the sensitivity of the actor output to different observation features. For an actor output component $a_{i,k}$ and an observation component $o_{i,m}$, the saliency magnitude can be written as

$$S_{i,m} = \left| \frac{\partial a_{i,k}}{\partial o_{i,m}} \right|. \quad (57)$$

For grouped features, such as LiDAR, goal information, robot pose, relative robot positions, and velocity, the group saliency can be computed as

$$S_i^{(g)} = \frac{1}{|\mathcal{G}_g|} \sum_{m \in \mathcal{G}_g} \left| \frac{\partial a_{i,k}}{\partial o_{i,m}} \right|, \quad (58)$$

where $\mathcal{G}_g$ is the set of observation indices belonging to feature group g .

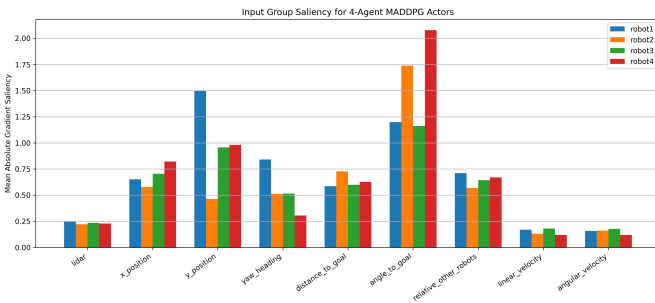


Fig. 4: Group saliency analysis of the trained actor policy. Larger values indicate observation groups that have stronger influence on the selected action.

Saliency analysis is useful because reinforcement learning policies can be difficult to interpret from reward curves alone. If the policy strongly depends on goal distance and goal angle, it suggests that the actor has learned goal-directed behavior. If LiDAR and relative robot features have significant influence, it suggests that the policy is using obstacle and inter-agent information for navigation.

B. Role of Centralized Training

The CTDE structure is an important part of the control design. If each robot were trained independently, the learning problem would become highly non-stationary because the other robots are also changing their policies. By using centralized critics, each agent can evaluate actions with knowledge of the global state and joint action. This improves the critic's ability to assign value to coordinated behavior. During execution, the actor only requires local information, which makes the trained policy more practical for decentralized deployment.

VI. CONCLUSION

This paper presented a MADDPG-based multi-agent reinforcement learning framework for autonomous navigation and collision avoidance of differential-drive unmanned ground vehicles. The navigation problem was formulated as a multi-agent continuous-control task, where each robot learns to generate linear and angular velocity commands from local observations. A centralized training and decentralized execution framework was used, allowing the critics to learn from the global state and joint actions during training while enabling each actor to operate using only local observations during execution. The four-robot Gazebo case study demonstrated the feasibility of applying MADDPG to multi-agent navigation with static obstacles, boundary constraints, and inter-robot collision avoidance.

Although the proposed framework demonstrated effective multi-agent navigation in simulation, further validation is needed under more diverse environments and testing conditions. Future work will focus on improving policy robustness, adding broader quantitative evaluation, and extending the framework toward more realistic deployment scenarios.

REFERENCES

- [1] N. R. Council, D. on Engineering, P. Sciences, N. S. Board, and C. on Autonomous Vehicles in Support of Naval Operations, *Autonomous vehicles in support of naval operations*. National Academies Press, 2005.
- [2] C. Wong, E. Yang, X.-T. Yan, and D. Gu, "An overview of robotics and autonomous systems for harsh environments," in *2017 23rd International Conference on Automation and Computing (ICAC)*. IEEE, 2017, pp. 1–6.
- [3] A. Pongpunwattana and R. Rysdyk, "Real-time planning for multiple autonomous vehicles in dynamic uncertain environments," *Journal of Aerospace Computing, Information, and Communication*, vol. 1, no. 12, pp. 580–604, 2004.
- [4] A. Bektemessov, "A survey of path planning and obstacle avoidance techniques in mobile robotics," *Engineering, Technology & Applied Science Research*, vol. 15, no. 6, pp. 29 632–29 640, 2025.
- [5] Z. Wei, S. Wang, K. Chen, and F. Wang, "Ros-based navigation and obstacle avoidance: A study of architectures, methods, and trends," *Sensors*, vol. 25, no. 14, p. 4306, 2025.

- [6] Y. Li, "Deep reinforcement learning: An overview," *arXiv preprint arXiv:1701.07274*, 2017.
- [7] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *Journal of artificial intelligence research*, vol. 4, pp. 237–285, 1996.
- [8] J. Du, Z. Kong, A. Sun, J. Kang, D. Niyato, X. Chu, and F. R. Yu, "Maddpg-based joint service placement and task offloading in mec empowered air-ground integrated networks," *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 10 600–10 615, 2023.
- [9] M. L. Puterman, "Markov decision processes," *Handbooks in operations research and management science*, vol. 2, pp. 331–434, 1990.
- [10] T. Kobayashi and W. E. L. Ilboudo, "T-soft update of target network for deep reinforcement learning," *Neural Networks*, vol. 136, pp. 63–71, 2021.